第2回：LLM APIの基礎

Pythonから gpt-6-luna を呼び出す

今日のゴールと流れ

APIへの入力と、返ってきた出力を説明できる。

時間	内容
10分	前回の復習、モデルとAPI
20分	HTTP、JSON、APIキー
20分	Pythonコードの読み方
30分	呼び出しと比較の演習
10分	利用量、エラー、提出

サービスの利用とAPIの利用

チャット画面では、人が入力し、画面に答えが表示されます。
APIでは、Pythonが入力を送り、Pythonが結果を受け取ります。

受け取った結果を画面に表示したり、他の処理へ渡したりできます。
チャット製品の契約とAPIの利用枠は、同じとは限りません。
講師の案内に従ってAPI利用環境を準備します。

HTTPとJSONの復習

要素	LLM APIでの意味
POST	入力を送り、回答の生成を依頼
認証ヘッダー	利用するアカウントのキーを渡す
JSONのリクエスト	モデル、指示、質問など
JSONのレスポンス	生成結果、状態、利用量など

Python SDKは、この通信を呼びやすくするライブラリです。

2種類のAPI

第3回以降は、APIが2か所に登場します。

API	呼ぶ側	受け取る側
自作の `/api/ask`	ブラウザ	自分のFastAPI
Responses API	自分のPython	外部のLLMサービス

今日は下の通信だけを扱います。Web画面はまだ作りません。

1問の処理を追う

質問：「FastAPIを2文で説明して」

順番	場所	起こること
1	one_shot.py	キーボードから質問を受け取る
2	llm.py	質問と指示をSDKへ渡す
3	外部API	入力を受け取り、生成結果を返す
4	llm.py	回答と利用量を辞書にまとめる
5	one_shot.py	辞書の内容を画面へ表示する

通信が発生するのは、SDKから外部APIへ依頼する部分です。

秘密情報の保管

APIキーを知る人は、利用権限を使える場合があります。

- Codespaces Secretsの名前は `OPENAI_API_KEY`
- このリポジトリへのアクセスを許可する
- 設定後はCodespacesを停止・再開する
- キーの値を画面、ログ、提出物に貼らない

漏れた場合はキーを無効化・再発行します。Gitから消すだけでは不十分です。

.envによる設定

Codespaces Secretsを使わない場合の方法です。

```
cd session02/exercise
cp .env.example .env
```

`.env` をエディタで開き、キーを設定します。
APIを使うときは `DEMO_MODE=false` に変更します。
`.env` はGitの対象外。環境変数が既にあれば、その値を優先します。

設定の確認と起動場所

```
source .venv/bin/activate
cd session02/exercise
python one_shot.py
```

- API演習では、講師の案内に従い `DEMO_MODE=false` にする
- デモなら固定文が返り、API呼び出しの穴埋めは通らない
- 環境変数がある場合、同名の `.env` の値より優先する

キーの値を表示して確認する必要はありません。

今回のファイル

ファイル	役割
`one_shot.py`	質問を入力し、結果を表示
`llm.py`	APIを呼び出す
`settings.py`	モードとモデル名を読む
`.env.example`	秘密を含まない設定例

読み順は `one_shot.py` 、 `ask_plain()` 、 `make_client()` です。

入口のコード

```
question = input('質問: ').strip()
result = ask_plain(question)
print(result['answer'])
print('入力トークン:', result['input_tokens'])
print('出力トークン:', result['output_tokens'])
```

関数に質問を渡し、戻り値の辞書から必要な項目を取り出します。

`strip()` は文字列の前後の空白を除きます。

関数に渡す値と戻る値

```
question = 'FastAPIを2文で説明して'
result = ask_plain(question)
answer = result['answer']
```

名前	型	意味
question	文字列	関数に渡す質問
result	辞書	回答と利用量のまとまり
answer	文字列	辞書から取り出した回答

辞書全体と、その中の回答文字列を区別します。

クライアントの作成

```
from openai import OpenAI

client = OpenAI(timeout=30.0, max_retries=0)
```

- `OPENAI_API_KEY` はSDKが環境変数から取得
- `timeout` は応答待ちの時間の設定
- `max_retries=0` は自動再試行をしない設定

教材では、意図しない再送による追加呼び出しを避けます。

Responses APIの呼び出し

```
response = client.responses.create(
    model=MODEL,  # settings.pyではgpt-6-luna
    reasoning={'effort': 'none'},
    instructions='日本語で簡潔に答えてください。',
    input=question,
    max_output_tokens=600,
    store=False,
)
```

教材は短い応答の観察から始めるため、推論設定を `none` にします。

指定した項目の意味

項目	意味
model	呼び出すモデルのID
instructions	アプリが与える指示
input	今回の質問や参考情報
max_output_tokens	出力の上限。推論トークンも含む
store	Responses APIでの応答保存の指定

`store=False` だけで、あらゆるサービス側ログが無くなるわけではありません。
秘密情報は送信しません。

指示と質問の組み合わせ

試行	instructions	input
A	初学者向けに2文で説明	FastAPIとは？
B	箇条書きで3項目に整理	FastAPIとは？

ここでは質問を固定して、説明の形式への影響を観察します。
モデル名や出力上限も固定し、変更した項目を記録します。

予想：形式を変えると、説明に含まれる情報も変わるでしょうか。

応答の取り出し

```
answer = response.output_text
input_tokens = response.usage.input_tokens
output_tokens = response.usage.output_tokens
```

SDKの `output_text` で回答のテキストを取り出します。
教材は `status` が完了か、本文が空でないかも確認します。
途中で切れた回答を、成功として表示しないためです。

API応答から教材の辞書へ

SDKで読む項目	教材が返すキー
response.output_text	answer
response.usage.input_tokens	input_tokens
response.usage.output_tokens	output_tokens

`response_result()` が、この対応で辞書を作ります。
呼び出し元は、API応答の詳細をすべて扱う必要がありません。
実際の利用量は、実行して得た値を記録します。

完了を確認する理由

応答の状態	教材の処理
completed で本文あり	回答と利用量を返す
completed 以外	ValueErrorで呼び出し元へ知らせる
本文が空白だけ	ValueErrorで呼び出し元へ知らせる

出力が途中で切れた場合も、正常な回答として表示しません。
質問や資料の長さを見直してから、再実行を判断します。

実習1：API呼び出しの穴埋め

1. `llm.py` の `TODO` を探す
2. 呼び出し部分のコメントを外す
3. `raise NotImplementedError(...)` を削除する
4. `python one_shot.py` を実行する
5. 「FastAPIを2文で説明して」と入力する

デモではAPI部分を通りません。穴埋めの確認は講師指定のAPI環境で行います。

実習1の完了確認

1. 実行したファイルが `session02/exercise/one_shot.py` か
2. `llm.py` の仮の `raise` を削除したか
3. APIモードで回答本文と入力・出力トークン数が表示されたか
4. その条件と結果をワークシートへ記録したか

API環境が使えない場合は、講師の実演を観察します。
自分のコードのAPI動作は「未検証」と区別して残します。

実習2：指示と質問の比較

同じ質問のまま、指示だけを変更します。

試行	指示の例
A	初学者向けに2文で説明する
B	箇条書きで3項目にまとめる
C	専門用語には短い説明を添える

内容・形式・入力と出力のトークン数を記録します。
同じ入力でも同じ結果を保証するとは限りません。

比較結果の読み取り

観点	観察する例
形式	指定した文数・項目数になったか
内容	同じ要点が含まれるか、条件が抜けていないか
用語	初学者が分からない用語に説明があるか
利用量	入力・出力のトークン数はどう変わったか

1回だけの差で断定せず、必要な例は同じ条件で再実行します。
追加の呼び出しにも利用量が発生するため、講師の予算内で行います。

利用料金の考え方

概算は、入力と出力のトークン数に、それぞれの単価を掛けます。

例：単価が100万トークン当たりの表示なら、

```
入力数 ÷ 1,000,000 × 入力単価
＋ 出力数 ÷ 1,000,000 × 出力単価
```

実際にはキャッシュ等の条件も関係します。
料金表と利用状況画面を講師と確認し、数値は開講時に更新します。

利用量の計算練習

説明用に、1問の入力を800、出力を200トークンとします。
同じ利用量の質問を10回送ると、入力8,000、出力2,000です。

100万トークン当たりの単価を、入力P円・出力Q円とすると：

概算料金 = 0.008 × P ＋ 0.002 × Q 円

数値は計算練習用で、実測値や現在の料金ではありません。
入力と出力の単価は別々に確認します。

よくあるエラー

状況	確認すること
キーが見つからない	Secretsの名前・対象リポジトリ・再起動
認証エラー	キーの有効性。値そのものは共有しない
モデルを使えない	`gpt-6-luna` の利用権限・モデル名
利用制限	レート制限・利用枠・請求設定
待ち時間が長い	ネットワーク・入力の長さ

モデルを勝手に変更せず、講師に状況を伝えます。

失敗した場所の切り分け

観察した状態	まず確認すること
importで止まる	仮想環境と依存パッケージ
NotImplementedError	穴埋めと仮のraise
デモの固定文だけが返る	DEMO_MODEと設定の優先順位
キー設定の案内が出る	Secretsの名前・許可・再起動
回答が完了しない	入力の長さと表示された案内

相談時は、操作手順とエラーの種類を伝えます。キーは共有しません。

Pythonの例外

```
try:
    result = ask_plain(question)
except ValueError as error:
    print(error)
```

`try` で処理し、失敗したときは `except` へ進みます。
次回は失敗をHTTPステータスと画面のメッセージに変換します。

確認問題

1. ブラウザのJavaScriptにキーを書いてよい？
2. 出力を短くしても、入力トークンは料金に関係する？
3. `instructions` と `input` は何を分けている？
4. デモモードのトークン数0は、無料のLLMを使った意味？

提出物と次回

提出：変更した `llm.py` と `worksheet.md` のGitHub URL。
APIキー、 `.env` 、APIの生の例外全文は提出しません。

```
git status
git add session02/exercise/llm.py session02/exercise/worksheet.md
git commit -m "第2回: API呼び出しと比較"
git push
```

次回は、同じ処理をWeb画面から呼び出します。